Penggalian Data
07

Sistem Informasi
Institut Teknologi Kalimantan

How to split other data types?

Numeric Data:

Weight	Heart Disease
220	Yes
180	Yes
225	Yes
190	No
155	No

How to determine best weight to divide the patients?

1. Sort the patients by weight, lowest to highest
2. Calculate the average weight of all adjacent patients
3. Calculate the Gini impurity for all average weights
4. Choose the average weight with lowest Gini impurity as the cutoff

How to split other data types?

Numeric Data:

	Weight	Heart Disease
Lowest	155	No
	180	Yes
	190	No
	220	Yes
Highest	225	Yes

Sort the patients by weight, lowest to highest

Weight	Heart Disease
155	No
167.5	
180	Yes
185	
190	No
205	
220	Yes
222.5	
225	Yes

Calculate the average weight of all adjacent patients

Weight	Heart Disease	
155	No	
167.5		Gini impurity = ?
180	Yes	
185		Gini impurity = ?
190	No	
205		Gini impurity = ?
220	Yes	
222.5		Gini impurity = ?
225	Yes	

Calculate the **Gini impurity** for all average weights

How to split other data types?

Numeric Data:

Weight	Heart Disease
155	No
167.5	
180	Yes
185	
190	No
205	
220	Yes
222.5	
225	Yes

Gini impurity = 1 - (probability of "yes")^2 - (probability of "no")^2

$$= 1 - \left(\frac{0}{0+1}\right)^2 - \left(\frac{1}{0+1}\right)^2$$

Gini impurity for Weight < 167.5

$$= \left(\frac{1}{1+4}\right) 0 + \left(\frac{4}{1+4}\right) 0.375 = 0.3$$

How to split other data types?

Numeric Data:

Weight	Heart Disease
155	No
167.5	
180	Yes
185	
190	No
205	
220	Yes
222.5	
225	Yes

Gini impurity = 0.3

Gini impurity = 0.47

Gini impurity = 0.27

Gini impurity = 0.4

Choose the average weight with lowest Gini impurity as the cutoff

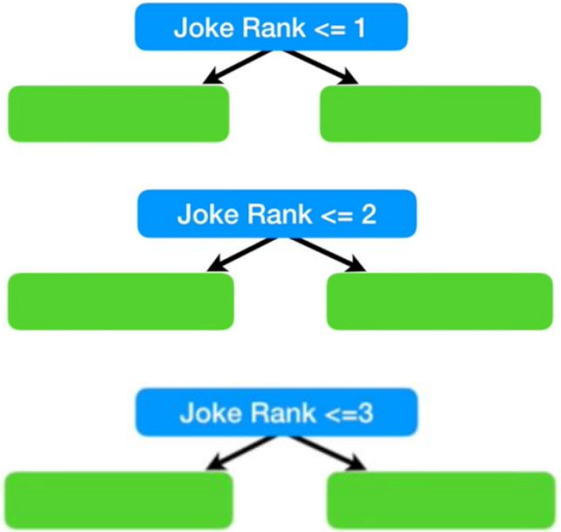
weight < 205

How to split other data types?

Ranked Data:

Rank my jokes...	Likes StatQuest
1	Yes
1	No
3	Yes
1	Yes
etc...	etc...

- Ranked data is similar to numeric data
- But we should calculate Gini impurity of all possible ranks

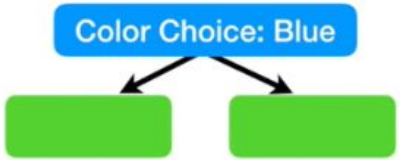


How to split other data types?

Multiple-choice data:

Color Choice	Likes StatQuest
Green	Yes
Blue	No
Red	Yes
Green	Yes
etc…	etc…

- For multiple-choice data, we should calculate all Gini impurity score for each choice AND each possible combination of choices



- Color Choice: Blue
- Color Choice: Green
- Color Choice: Red
- Color Choice: Blue and Green
- Color Choice: Blue and Red
- Color Choice: Red and Green

Iterative Dichotomiser 3 (ID3)

- ID3 algorithm allows multiway splits (two or more branches)
- The process is similar to using CART algorithm except for the method for splitting/measuring impurity
- To measure purity in ID3, we use **Entropy** (value is between 0 to 1)

$$H(S) = -\sum_{i=1}^{c} p_i \log_2(p_i)$$

- However, ID3 only works for categorical attributes
- The improvement of ID3 algorithm is called C4.5

- After calculating Entropy for each subset, we can calculate the Information Gain for the attribute

$$Gain(S, A) = H(S) - \sum_{V \epsilon Values(A)} \frac{|S_V|}{|S|} H(S_V)$$

V = possible values of A

S = set of examples {S}

S_V = subset where X_A = V

- We can choose the attribute with the **highest Information Gain as the splitting attribute**

Iterative Dichotomiser 3 (ID3)

ID3 Algorithm:

1. Calculate the **entropy** of the dataset (root node)
2. For each attribute:
   - Calculate the **information gain** based on that attribute
3. Select the attribute with the highest information gain as the **splitter**
4. Divide the dataset into subsets based on the selected attribute values
5. Repeat steps 1–4 for each subset until:
   - All instances in the subset have the same class, or
   - There are no attributes left

Day	Outlook	Temp	Humidity	Wind	PlayTennis
D1	Sunny	Hot	High	Weak	No
D2	Sunny	Hot	High	Strong	No
D3	Overcast	Hot	High	Weak	Yes
D4	Rain	Mild	High	Weak	Yes
D5	Rain	Cool	Normal	Weak	Yes
D6	Rain	Cool	Normal	Strong	No
D7	Overcast	Cool	Normal	Strong	Yes
D8	Sunny	Mild	High	Weak	No
D9	Sunny	Cool	Normal	Weak	Yes
D10	Rain	Mild	Normal	Weak	Yes
D11	Sunny	Mild	Normal	Strong	Yes
D12	Overcast	Mild	High	Strong	Yes
D13	Overcast	Hot	Normal	Weak	Yes
D14	Rain	Mild	High	Strong	No

Iterative Dichotomiser 3 (ID3)

1. Calculate the **entropy** of the dataset (root node)

   Total = 14 records
   Yes = 9
   No = 5

$$H(S) = -\frac{9}{14}\log_2\frac{9}{14} - \frac{5}{14}\log_2\frac{5}{14}$$
$$\approx 0.94$$

Day	Outlook	Temp	Humidity	Wind	PlayTennis
D1	Sunny	Hot	High	Weak	No
D2	Sunny	Hot	High	Strong	No
D3	Overcast	Hot	High	Weak	Yes
D4	Rain	Mild	High	Weak	Yes
D5	Rain	Cool	Normal	Weak	Yes
D6	Rain	Cool	Normal	Strong	No
D7	Overcast	Cool	Normal	Strong	Yes
D8	Sunny	Mild	High	Weak	No
D9	Sunny	Cool	Normal	Weak	Yes
D10	Rain	Mild	Normal	Weak	Yes
D11	Sunny	Mild	Normal	Strong	Yes
D12	Overcast	Mild	High	Strong	Yes
D13	Overcast	Hot	Normal	Weak	Yes
D14	Rain	Mild	High	Strong	No

Iterative Dichotomiser 3 (ID3)

2. For each attribute:
   - Calculate the **information gain** based on that attribute

OUTLOOK:

- Sunny: 5 records → Yes = 2, No = 3 →

$$H(\text{Sunny}) = -\frac{2}{5}\log_2\frac{2}{5} - \frac{3}{5}\log_2\frac{3}{5} \approx 0.97$$

- Overcast: 4 records → Yes = 4, No = 0 →

$$H(\text{Overcast}) = -1\log_2 1 = 0.00$$

- Rain: 5 records→ Yes = 3, No = 2 →

$$H(\text{Rain}) \approx 0.97$$

Day	Outlook	Temp	Humidity	Wind	PlayTennis
D1	Sunny	Hot	High	Weak	No
D2	Sunny	Hot	High	Strong	No
D3	Overcast	Hot	High	Weak	Yes
D4	Rain	Mild	High	Weak	Yes
D5	Rain	Cool	Normal	Weak	Yes
D6	Rain	Cool	Normal	Strong	No
D7	Overcast	Cool	Normal	Strong	Yes
D8	Sunny	Mild	High	Weak	No
D9	Sunny	Cool	Normal	Weak	Yes
D10	Rain	Mild	Normal	Weak	Yes
D11	Sunny	Mild	Normal	Strong	Yes
D12	Overcast	Mild	High	Strong	Yes
D13	Overcast	Hot	Normal	Weak	Yes
D14	Rain	Mild	High	Strong	No

Iterative Dichotomiser 3 (ID3)

2. For each attribute:
   - Calculate the **information gain** based on that attribute

OUTLOOK:

- Weighted average entropy:

$$\frac{5}{14} \times 0.97 + \frac{4}{14} \times 0.00 + \frac{5}{14} \times 0.97 =$$

$$= \frac{10}{14} \times 0.97 \approx 0.69$$

- Information Gain

$$IG(\text{Outlook}) = 0.94 - 0.69 = \mathbf{0.25}$$

Day	Outlook	Temp	Humidity	Wind	PlayTennis
D1	Sunny	Hot	High	Weak	No
D2	Sunny	Hot	High	Strong	No
D3	Overcast	Hot	High	Weak	Yes
D4	Rain	Mild	High	Weak	Yes
D5	Rain	Cool	Normal	Weak	Yes
D6	Rain	Cool	Normal	Strong	No
D7	Overcast	Cool	Normal	Strong	Yes
D8	Sunny	Mild	High	Weak	No
D9	Sunny	Cool	Normal	Weak	Yes
D10	Rain	Mild	Normal	Weak	Yes
D11	Sunny	Mild	Normal	Strong	Yes
D12	Overcast	Mild	High	Strong	Yes
D13	Overcast	Hot	Normal	Weak	Yes
D14	Rain	Mild	High	Strong	No

Iterative Dichotomiser 3 (ID3)

2. For each attribute:
   - Calculate the **information gain** based on that attribute

HUMIDITY:

$$IG(\text{Humidity}) = 0.94 - 0.79 \approx \mathbf{0.15}$$

WIND:

$$IG(\text{Wind}) = 0.94 - 0.89 \approx \mathbf{0.05}$$

TEMP:

$$IG(\text{Temperature}) = 0.94 - 0.91 \approx \mathbf{0.03}$$

Day	Outlook	Temp	Humidity	Wind	PlayTennis
D1	Sunny	Hot	High	Weak	No
D2	Sunny	Hot	High	Strong	No
D3	Overcast	Hot	High	Weak	Yes
D4	Rain	Mild	High	Weak	Yes
D5	Rain	Cool	Normal	Weak	Yes
D6	Rain	Cool	Normal	Strong	No
D7	Overcast	Cool	Normal	Strong	Yes
D8	Sunny	Mild	High	Weak	No
D9	Sunny	Cool	Normal	Weak	Yes
D10	Rain	Mild	Normal	Weak	Yes
D11	Sunny	Mild	Normal	Strong	Yes
D12	Overcast	Mild	High	Strong	Yes
D13	Overcast	Hot	Normal	Weak	Yes
D14	Rain	Mild	High	Strong	No

Iterative Dichotomiser 3 (ID3)

3. Select the attribute with the highest information gain as the splitter

Bandingkan nilai IG (dibulatkan):

- IG(Outlook) = 0.25
- IG(Humidity) = 0.15
- IG(Wind) = 0.05
- IG(Temperature) = 0.03

Outlook memiliki information gain tertinggi → **Outlook dipilih sebagai root.**

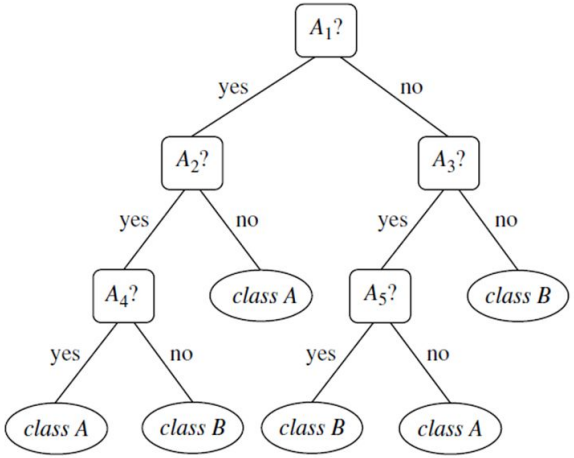
Day	Outlook	Temp	Humidity	Wind	PlayTennis
D1	Sunny	Hot	High	Weak	No
D2	Sunny	Hot	High	Strong	No
D3	Overcast	Hot	High	Weak	Yes
D4	Rain	Mild	High	Weak	Yes
D5	Rain	Cool	Normal	Weak	Yes
D6	Rain	Cool	Normal	Strong	No
D7	Overcast	Cool	Normal	Strong	Yes
D8	Sunny	Mild	High	Weak	No
D9	Sunny	Cool	Normal	Weak	Yes
D10	Rain	Mild	Normal	Weak	Yes
D11	Sunny	Mild	Normal	Strong	Yes
D12	Overcast	Mild	High	Strong	Yes
D13	Overcast	Hot	Normal	Weak	Yes
D14	Rain	Mild	High	Strong	No

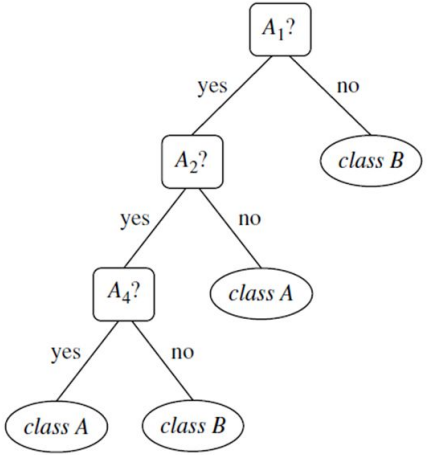
Decision Tree Pruning

- **Pruning** is a process of deleting unnecessary nodes from a tree to get the **optimal decision tree**

- Pruned trees tend to be smaller and less complex (easier to understand)
- They are usually faster and better at correctly classifying independent test data (i.e., of previously unseen tuples)

- Two kinds of pruning:
  - **Pre-pruning** → Done while building decision tree
  - **Post-pruning** → Done after the decision tree is built





Pre-Pruning (Early Stopping)

- Stop growing the tree early.
- Conditions:
  - Minimum number of samples per node.
  - Maximum tree depth.
  - Minimum information gain threshold.

For example:

- While building, stop splitting if:
  - Information Gain < 0.01
  - Node has < 5 samples
  - Tree depth = 5

- **Pros:** faster, simpler
- **Cons:** may underfit if stopped too early.

Post-Pruning

- Grow the full tree first.
- Then remove branches that do not improve validation accuracy.

Techniques:

- Reduced Error Pruning (REP)
  Remove nodes if performance on validation data does not drop.

- Cost Complexity Pruning (CCP) (used in CART)
  Minimize this cost-function:

$$R_\alpha(T) = R(T) + \alpha \times |T|$$

R(T) : error of the tree
|T| : number of leaf nodes
α : complexity parameter (penalty for large trees)

Decision Tree Pruning

Contoh Post-Pruning:

- Misalkan ada decision tree:

Subtree	#Leaves	Training Error (R(T))
T0 (Full tree)	6	0.05
T1 (Remove 1 branch)	4	0.07
T2 (Smaller tree)	2	0.10
T3 (Root only)	1	0.30

Ide utama cost-complexity pruning:

- cari pohon yang punya keseimbangan antara kompleksitas (jumlah daun) dan kesalahan (impurity)

  Minimalkan:

$$R_\alpha(T) = R(T) + \alpha \times |T|$$

α = 0 artinya tidak ada penalty (pohon asli tanpa pruning)

α semakin besar, maka semakin besar penalti (kehilangan kompleksitas)

Decision Tree Pruning

Contoh Post-Pruning:

- Hitung cost-complexity

α	Tree	R(T)	|T|	Rα(T) = R(T) + α|T|
0.00	T0	0.05	6	0.05
0.01	T0	0.05	6	0.11
0.01	T1	0.07	4	0.11
0.01	T2	0.10	2	0.12
0.05	T1	0.07	4	0.27
0.05	T2	0.10	2	0.20
0.05	T3	0.30	1	0.35

- Pilih Subtree dengan Nilai Rα(T) terkecil
- Saat α = 0.01 → 0.11 (T1 lebih sederhana)
- Saat α = 0.05 → 0.20 (T2)

- α mengontrol trade-off antara akurasi dan kompleksitas

- α kecil → biarkan tree lebih besar (low bias, high variance)
- α besar → potong tree lebih banyak (high bias, low variance)

Step 1 — Import libraries & load dataset

```
from sklearn.tree import DecisionTreeClassifier, plot_tree
from sklearn.datasets import load_breast_cancer
from sklearn.model_selection import train_test_split
import matplotlib.pyplot as plt
import numpy as np
```

Step 2 — Load dataset & split data

```
# Load the Breast Cancer dataset
X, y = load_breast_cancer(return_X_y=True)
feature_names = load_breast_cancer().feature_names
target_names = load_breast_cancer().target_names

# Split data into training and testing sets
X_train, X_test, y_train, y_test = train_test_split(X, y, random_state=42)
```

Step 3 — Train the full (unpruned) Decision Tree

```
# Train a full Decision Tree
clf_full = DecisionTreeClassifier(random_state=0)
clf_full.fit(X_train, y_train)

# Accuracy before pruning
acc_before = clf_full.score(X_test, y_test)
print(f"Accuracy before pruning: {acc_before:.3f}")
```

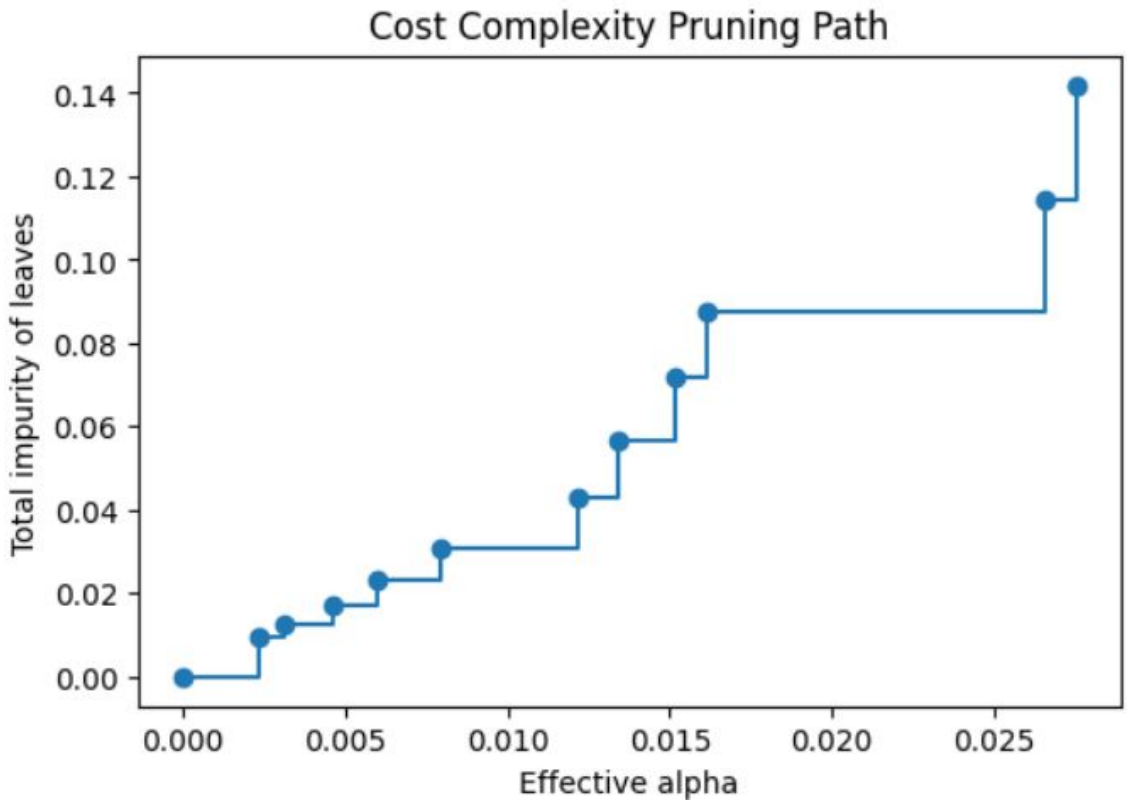
OUTPUT: (Test accuracy)

```
Accuracy before pruning: 0.930
```

Step 4 — Get cost complexity pruning path

```
# Compute the effective alphas and corresponding total leaf impurities
path = clf_full.cost_complexity_pruning_path(X_train, y_train)
ccp_alphas, impurities = path.ccp_alphas, path.impurities

# Visualize alpha vs total impurity
plt.figure(figsize=(6, 4))
plt.plot(ccp_alphas[:-1], impurities[:-1], marker="o",
drawstyle="steps-post")
plt.xlabel("Effective alpha")
plt.ylabel("Total impurity of leaves")
plt.title("Cost Complexity Pruning Path")
plt.show()
```

Step 4 — Get cost complexity pruning path



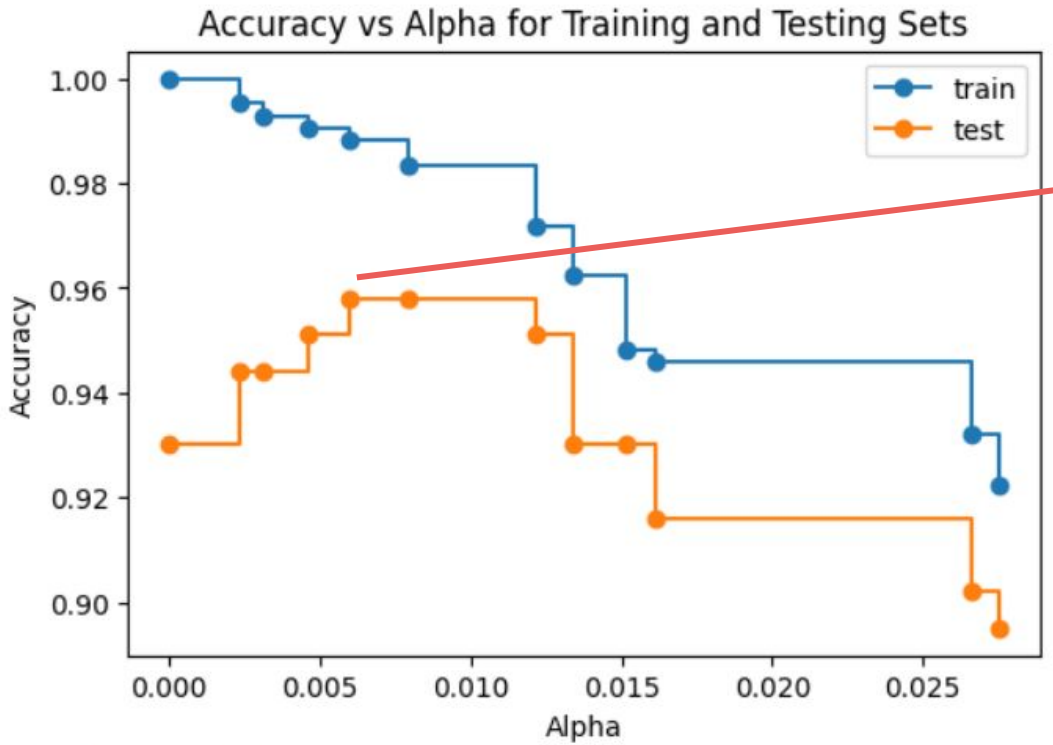
Step 5 — Train trees for each alpha & find the best one

```
clfs = []
for ccp_alpha in ccp_alphas:
    clf = DecisionTreeClassifier(random_state=0, ccp_alpha=ccp_alpha)
    clf.fit(X_train, y_train)
    clfs.append(clf)

# Remove the last (fully pruned) tree
clfs = clfs[:-1]
ccp_alphas = ccp_alphas[:-1]

# Compute accuracy for each alpha
train_scores = [clf.score(X_train, y_train) for clf in clfs]
test_scores = [clf.score(X_test, y_test) for clf in clfs]
```

Step 5 — Train trees for each alpha & find the best one

```
# Plot train vs test accuracy
plt.figure(figsize=(6, 4))
plt.plot(ccp_alphas, train_scores, marker="o", label="train",
drawstyle="steps-post")
plt.plot(ccp_alphas, test_scores, marker="o", label="test",
drawstyle="steps-post")
plt.xlabel("Alpha")
plt.ylabel("Accuracy")
plt.title("Accuracy vs Alpha for Training and Testing Sets")
plt.legend()
plt.show()

# Choose alpha with best test accuracy
best_alpha = ccp_alphas[np.argmax(test_scores)]
print(f"Best alpha = {best_alpha:.5f}")
```

Step 5 — Train trees for each alpha & find the best one



Best alpha ketika test accuracy tertinggi dan training accuracy tertinggi

```
Best alpha = 0.00596
```

Step 6 — Train pruned tree with best alpha

```
clf_pruned = DecisionTreeClassifier(random_state=0, ccp_alpha=best_alpha)
clf_pruned.fit(X_train, y_train)

# Accuracy after pruning
acc_after = clf_pruned.score(X_test, y_test)
print(f"Accuracy after pruning: {acc_after:.3f}")
```

OUTPUT:

```
Accuracy after pruning: 0.958
```

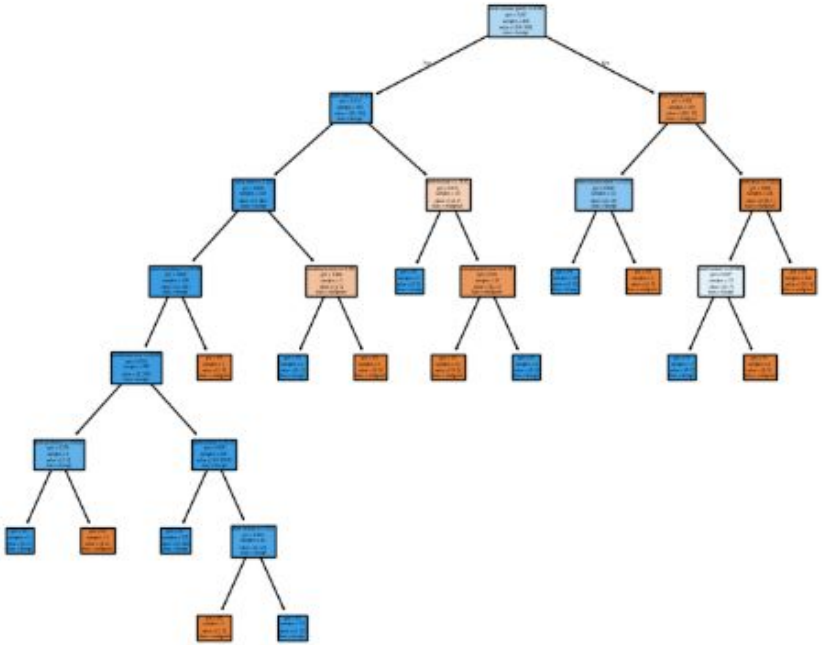
- Test accuracy meningkat dari sebelum pruning
- Sebelum pruning, pohon terlalu besar (overfit), sehingga training accuracytinggi, tapi test accuracylebih rendah
- Setelah pruning, pohon lebih general, sehingga meningkatkan test accuracy

Step 7 — Visualize full vs pruned tree

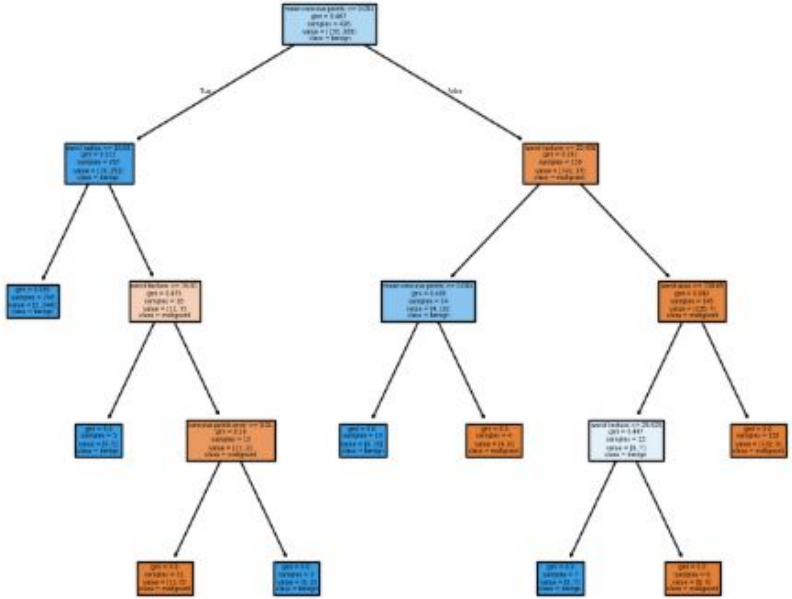
```
plt.figure(figsize=(14, 6))

plt.subplot(1, 2, 1)
plot_tree(clf_full, filled=True, feature_names=feature_names,
class_names=target_names)
plt.title("Full Tree")

plt.subplot(1, 2, 2)
plot_tree(clf_pruned, filled=True, feature_names=feature_names,
class_names=target_names)
plt.title(f"Pruned Tree (alpha={best_alpha:.5f})")

plt.tight_layout()
plt.show()
```

Step 7 — Visualize full vs pruned tree

Full Tree



Pruned Tree (alpha=0.00596)



Confusion Matrix

- A **confusion matrix** is a table that evaluates the performance of a classification model by comparing the model's predictions to the actual, true values

Components of a confusion matrix

- **True Positive (TP)**
- **True Negative (TN)**
- **False Positive (FP):** The model incorrectly predicted a positive outcome (e.g., it flagged a non-spam email as spam). This is also known as a Type I error.
- **False Negative (FN):** The model incorrectly predicted a negative outcome (e.g., it failed to flag a spam email). This is also known as a Type II error.

		PREDICTED CLASS	
ACTUAL CLASS		Class=Yes	Class=No
	Class=Yes	a	b
	Class=No	c	d

a: TP (true positive)

b: FN (false negative)

c: FP (false positive)

d: TN (true negative)

Accuracy

- Classification **accuracy** is the ratio of correct predictions to the total number of predictions

		PREDICTED CLASS	
ACTUAL CLASS		Class=Yes	Class=No
	Class=Yes	a (TP)	b (FN)
	Class=No	c (FP)	d (TN)

$$\text{Accuracy} = \frac{a+d}{a+b+c+d}$$

$$= \frac{TP+TN}{TP+TN+FP+FN}$$

Error Rate = 1 — Accuracy

Which model is better?

A

		PREDICTED	
ACTUAL		Class=Yes	Class=No
	Class=Yes	0	10
	Class=No	0	990

Accuracy: 99%

B

		PREDICTED	
ACTUAL		Class=Yes	Class=No
	Class=Yes	10	0
	Class=No	500	490

Accuracy: 50%

Model A

- If the context is a balanced dataset, where false positives and false negatives are equally bad:

  → then overall accuracy matters more

Model B

- This model is very aggressive in predicting positives (it catches all positives but generates many false alarms)
- If the context is rare-event detection (e.g., disease diagnosis, fraud detection, spam filtering):

  → we usually care more about Recall for the positive class

Problem with Accuracy as Measurement

Consider a 2-class problem:

- Number of Class NO examples = 990
- Number of Class YES examples = 10

→ If a model predicts everything to be class NO, accuracy is 990/1000 = 99 %

- This is misleading because this trivial model does not detect any class YES example
- Detecting the rare class is usually more interesting (e.g., frauds, intrusions, defects, etc)

		PREDICTED CLASS	
		Class=Yes	Class=No
ACTUAL CLASS	Class=Yes	0	10
	Class=No	0	990

Problem with Accuracy as Measurement

Lots of classification problems where the classes are skewed (more records from one class than another)

- Credit card fraud
- Intrusion detection
- Defective products in manufacturing assembly line

Evaluation measures such as accuracy are not well-suited for imbalanced class

		PREDICTED CLASS	
ACTUAL CLASS		Class=Yes	Class=No
	Class=Yes	0	10
	Class=No	0	990

Precision, Recall, F-measure

Precision (Positive Predictive Value):

- "Of all predicted positives, how many are correct?"

Recall (Sensitivity or True Positive Rate):

- "Of all actual positives, how many did we catch?"

	PREDICTED CLASS		
ACTUAL CLASS		Class=Yes	Class=No
	Class=Yes	a	b
	Class=No	c	d

$$\text{Precision (p)} = \frac{a}{a+c}$$

$$\text{Recall (r)} = \frac{a}{a+b}$$

$$\text{F - measure (F)} = \frac{2rp}{r+p} = \frac{2a}{2a+b+c}$$

Precision, Recall, F-measure

F1-Score / F-measure

- Harmonic mean of Precision and Recall
  - A model with high precision but low recall, or vice versa, will result in a lower F1-score compared to a model with moderately high values for both
- The score ranges from 0 to 1 (1 is the perfect precision and recall)
- Useful when:
  - Class distribution is imbalanced
  - You need a single performance measure

	PREDICTED CLASS		
ACTUAL CLASS		Class=Yes	Class=No
	Class=Yes	a	b
	Class=No	c	d

$$\text{Precision (p)} = \frac{a}{a+c}$$

$$\text{Recall (r)} = \frac{a}{a+b}$$

$$\text{F - measure (F)} = \frac{2rp}{r+p} = \frac{2a}{2a+b+c}$$

Specificity and Balanced Accuracy

Specificity (True Negative Rate):

- "Of all predicted negatives, how many are correct?"

$$\text{Specificity} = \frac{TN}{TN + FP}$$

	PREDICTED CLASS		
ACTUAL CLASS		Class=Yes	Class=No
	Class=Yes	a	b
	Class=No	c	d

Balanced Accuracy:

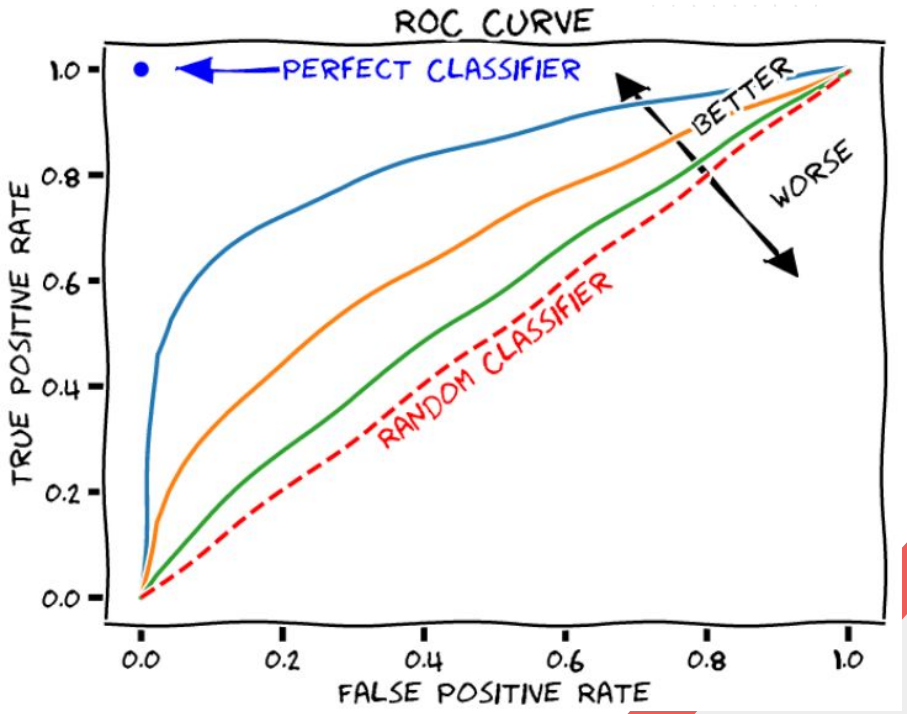
- Similar with F1-score, balanced accuracy is useful for imbalanced datasets

$$\text{Balanced Accuracy} = \frac{\text{Recall} + \text{Specificity}}{2}$$

ROC-Curve

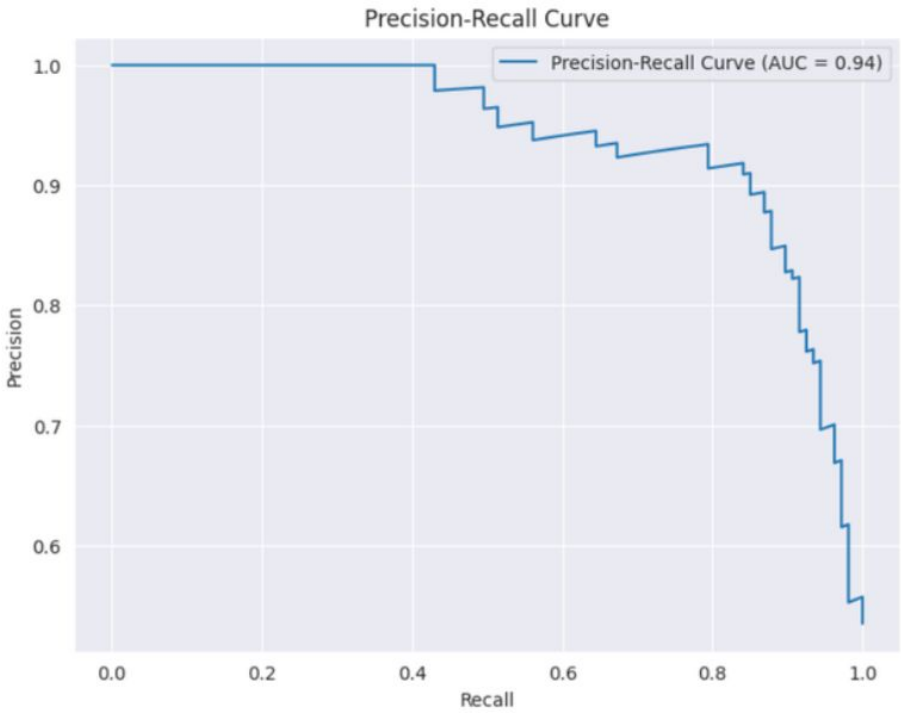
Receiver Operating Characteristic (ROC) curve:

- plots TPR (recall/sensitivity) vs. FPR across thresholds
- FPR = 1 - recall
- AUC (Area Under the Curve):
  - Closer to 1 → better model
  - 0.5 → random guessing



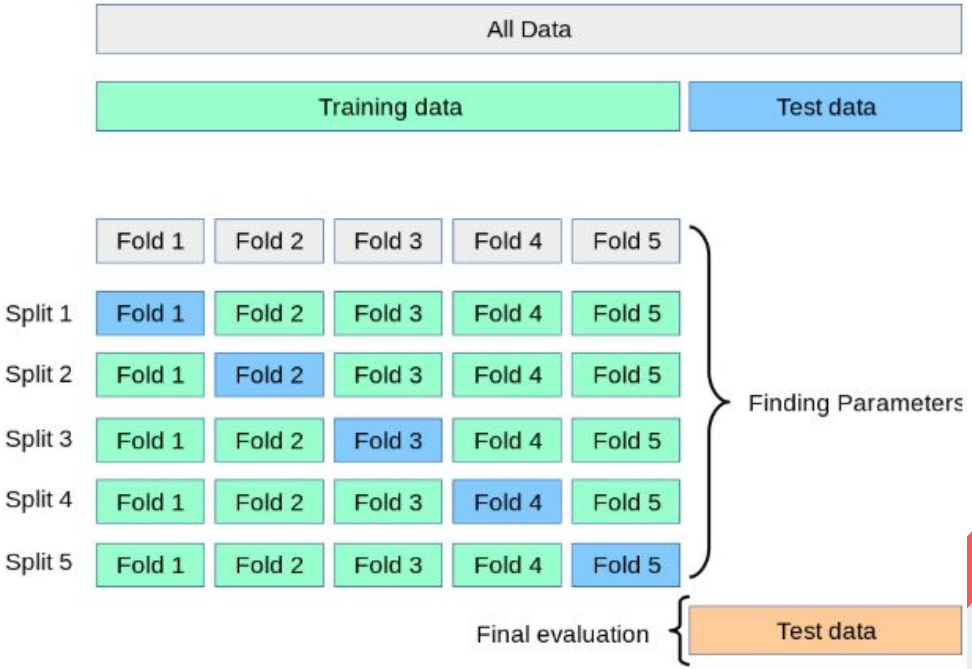
Precision-Recall Curve

- Shows trade-off between Precision and Recall
- A model with a high-performing curve stays closer to the top-right corner (high precision and high recall)
- High area under PR curve → good performance



Cross-Validation for Evaluation

- Split dataset into k folds
- Train on (k−1) folds, test on 1
- Repeat k times, average the results
- Reduces variance due to data partitioning



Rule-Based Classifier

- Classify records by using a collection of “if…then…” rules
- Each rule represents a pattern in the data.

- Rule: (Condition) → y
  where
  - Condition is a conjunction of tests on attributes
  - y is the class label

Examples
- (Blood Type=Warm) ∧ (Lay Eggs=Yes) → Birds
- (Taxable Income < 50K) ∧ (Refund=Yes) → Evade=No

Antecedent

Consequent

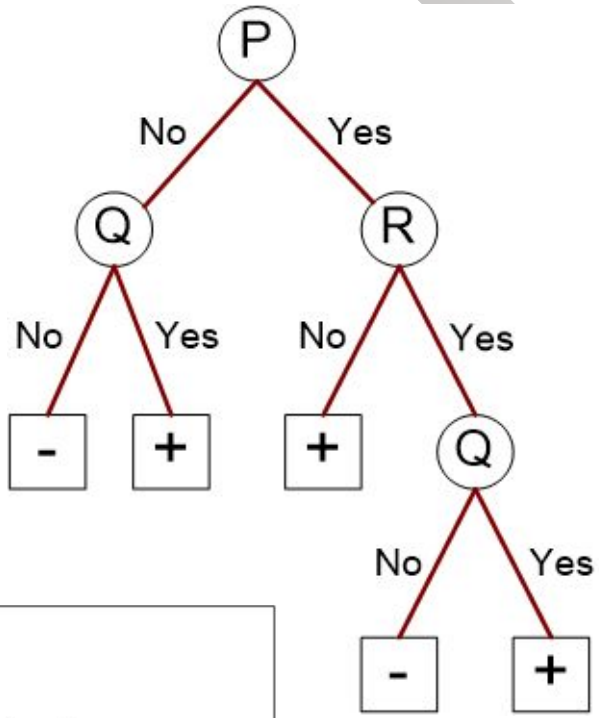
Similar characteristics as Decision Trees:
- Easy to interpret
- Naturally handles categorical data
- Rules are modular → easy to add or remove
- Performance comparable to DT

Rule-Based Classification Process

1. **Rule Generation:**
   Extract rules from training data (often from decision trees or directly via algorithms)

2. **Rule Ordering/Selection:**
   Determine which rule to apply first

3. **Classification:**
   Apply the first rule that matches the instance



Rule Set

r1: (P=No,Q=No) ==> -
r2: (P=No,Q=Yes) ==> +
r3: (P=Yes,R=No) ==> +
r4: (P=Yes,R=Yes,Q=No) ==> -
r5: (P=Yes,R=Yes,Q=Yes) ==> +

Rule Ordering Strategies

Ordered Rule Set (Decision List):

- Rules are applied in sequence until one matches
- The order is based on the quality of the rule
- This quality can be determined by metrics like accuracy or coverage
- Rules with higher quality are given higher priority

- Algorithms: RIPPER, OneR, CN2

Advantages:

- Simple and fast inference
- Easy interpretability

Disadvantages:

- Rule order is crucial
- Simple conflict resolution → if two rules could apply, only the first rule is used

Rule Ordering Strategies

Unordered Rule Set:

- Rules are independent
- Multiple rules may apply → use voting or weighting

- Algorithm: CBA (Classification Based on Associations)

Advantages:

- Flexible and robust
- Better coverage

Disadvantages:

- Requires conflict resolution
- Slower (all rules needs to be evaluated)
- Less interpretable

For example:

Rule	Condition	Prediction	Accuracy (Weight)
R1	(income > 50jt)	Approved	0.9
R2	(age < 25) ∧ (loan_ammount > 100jt)	Reject	0.8
R3	(has_prev_default == True)	Reject	0.95

New data:
income = 60jt, age = 22, loan_amt = 120jt, has_prev_default = False
Prediction = ?

Rule Ordering Strategies

Rule	Condition	Prediction	Accuracy (Weight)
R1	(income > 50jt)	Approved	0.9
R2	(age < 25) ∧ (loan_ammount > 100jt)	Reject	0.8
R3	(has_prev_default == True)	Reject	0.95

New data:
income = 60jt, age = 22, loan_amt = 120jt,
has_prev_default = False
Prediction = ?

1. Which rules match the new data?
   R1 matches → 1 vote Approved
   R2 matches → 1 vote Rejected

2. Voting → **Vote is tied**

3. Because the vote is tied, we use **weighted vote (cumulative weight)** for every matching rule

   R1 → Approved += 0.9
   R2 → Rejected += 0.8

   Approved wins in total.

Rule Coverage and Accuracy

Coverage of a rule:

- Fraction of records that satisfy the antecedent of a rule

$$\text{Coverage}(R_i) = \frac{\text{Number of instances that satisfy antecedent}}{\text{Total instances}}$$

Accuracy of a rule:

- Fraction of records that satisfy the antecedent that also satisfy the consequent of a rule

$$\text{Accuracy}(R_i) = \frac{\text{Number of correctly classified instances}}{\text{Number of instances that satisfy antecedent}}$$

Tid	Refund	Marital Status	Taxable Income	Class
1	Yes	Single	125K	No
2	No	Married	100K	No
3	No	Single	70K	No
4	Yes	Married	120K	No
5	No	Divorced	95K	Yes
6	No	Married	60K	No
7	Yes	Divorced	220K	No
8	No	Single	85K	Yes
9	No	Married	75K	No
10	No	Single	90K	Yes

High coverage + high accuracy = strong rule

Rule: (Status=Single) → No
Coverage = 40%, Accuracy = 50%

Extracting rules from decision tree

```
from sklearn.tree import DecisionTreeClassifier, export_text
from sklearn.datasets import load_iris
from sklearn.model_selection import train_test_split

X, y = load_iris(return_X_y=True)
X_train, X_test, y_train, y_test = train_test_split(X, y, random_state=42)

clf = DecisionTreeClassifier(max_depth=3, random_state=0)
clf.fit(X_train, y_train)

print(export_text(clf, feature_names=load_iris().feature_names))
```

Extracting rules from decision tree

Output:

```
|--- petal width (cm) <= 0.80
|   |--- class: 0
|--- petal width (cm) >  0.80
|   |--- petal length (cm) <= 4.75
|   |   |--- petal width (cm) <= 1.65
|   |   |   |--- class: 1
|   |   |--- petal width (cm) >  1.65
|   |   |   |--- class: 2
|   |--- petal length (cm) >  4.75
|   |   |--- petal width (cm) <= 1.75
|   |   |   |--- class: 1
|   |   |--- petal width (cm) >  1.75
|   |   |   |--- class: 2
```

Rule 1: (petal width <= 0.80) → class = 0

Rule 2: (petal width > 0.8) ∧ (petal length <= 4.75) ∧ (petal width <= 1.65) → class 1

Rule 3: (petal width > 0.8) ∧ (petal length <= 4.75) ∧ (petal width > 1.65) → class 2

Rule 4: (petal width > 0.8) ∧ (petal length > 4.75) ∧ (petal width <= 1.75) → class 1

Rule 5: (petal width > 0.8) ∧ (petal length > 4.75) ∧ (petal width > 1.75) → class 2

Any Questions?

Ujian Tengah Semester

Detail UTS:

- 10 soal pilihan ganda dan 3 soal essay dengan pengerjaan 90 menit (7.30 - 9.00 WITA)
- UTS dimulai serentak, tidak ada penambahan waktu bagi yang terlambat
- Selama UTS, hanya diperbolehkan menggunakan **cheat sheet** pribadi (1 lembar A4)
- Persiapkan alat tulis pribadi (kertas jawaban akan disediakan)

Peraturan:

- **DILARANG:**
  - Berdiskusi tentang soal/jawaban dengan siapapun
  - Membuka device elektronik selama UTS berlangsung

- Jika ditemukan melakukan pelanggaran, nilai UTS = 0
- Boleh meninggalkan ruang kelas **dengan menjaga ketenangan** kalau sudah menyerahkan lembar jawaban sebelum 9.00 WITA

- Presensi akan di-rekap Dosen berdasarkan Tanda Tangan Lembar Kehadiran

THANKS!

CREDITS: This presentation template was created by **Slidesgo**, and includes icons by **Flaticon**, and infographics & images by **Freepik**

Please keep this slide for attribution